

Source Data Model Implementation



Na vandaag...

Weet/ken je...

Kan je...

De juiste CREATE-TABLE-
volgorde bij het aanmaken van
Source Data Modellen

De datastructuur van een
Source Data Model
implementeren in SQLite.

Op welke 4 manieren je
SQLite-databases kan vullen
met behulp van Python

Welke criteria je tegen elkaar
moet afwegen bij het vullen
van Source Data Modellen

Welke logging-niveaus worden
onderscheiden door de
“loguru” library in Python

HC + WC



Source Data Modellen vullen
met data en de gekozen
inlaadstrategie verantwoorden

Logging aanbrengen in
Pythonscripts die Source Data
Modellen vullen vanuit
operationele databases

Globale fasering

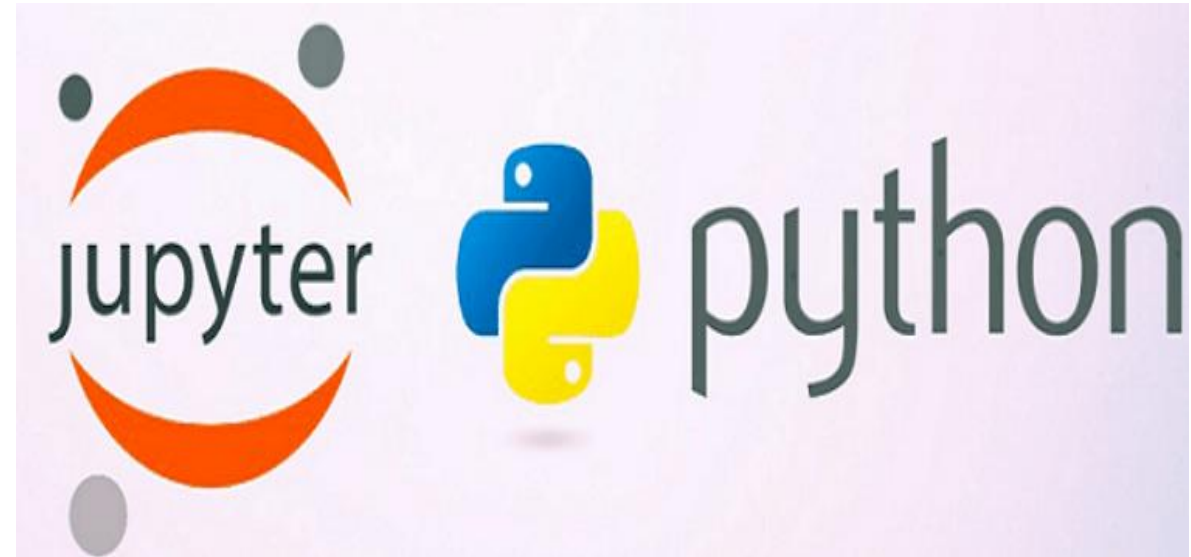
Stap 1: Nieuwe SDM-database



De datastructuur bouwen:

- Met SQL-CREATEs maak je SDM-tabellen en laat je ze naar elkaar verwijzen, middels primaire-vreemde-sleutelrelaties
- Deze tabellen zijn nu nog leeg

Stap 2: Python (in Jupyter Notebook)

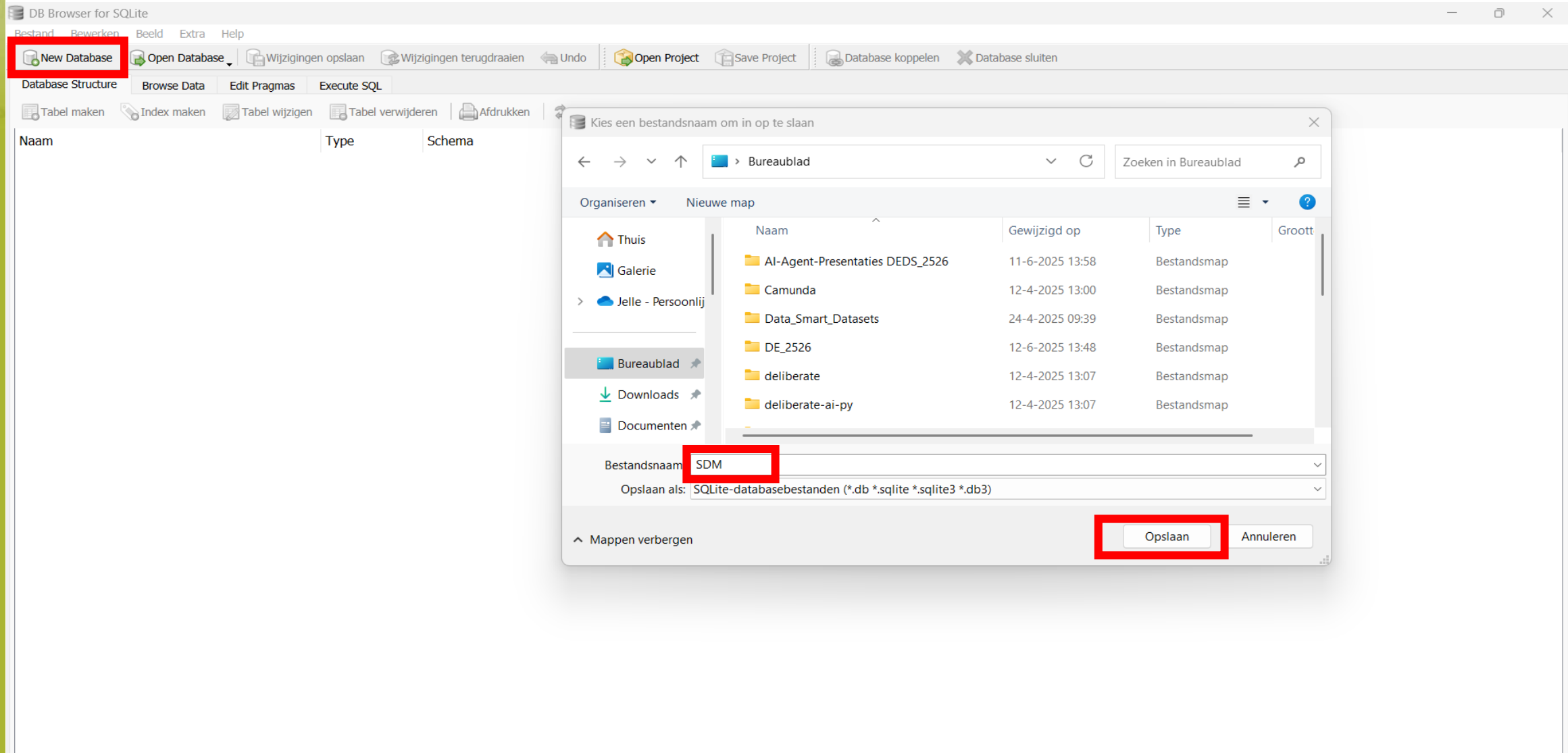


De datastructuur vullen:

- Je legt de connectie met operationele databronnen en het SDM. Beiden staan in SQLite.
- Tabelrijen uit operationele databronnen vertaal je naar INSERT INTO's, die je vervolgens toepast op het SDM.

Nieuwe database aanmaken

Het geïmplementeerde Source Data Model komt in SQLite in een nieuwe database te staan



De tabelstructuur uit brondatabases overnemen in SDM

```
CREATE TABLE Customer (  
  id INT,  
  fname TEXT,  
  lname TEXT,  
  address TEXT,  
  city TEXT,  
  state INT NOT NULL,  
  
  PRIMARY KEY(id),  
  FOREIGN KEY(state) REFERENCES State(state_id)  
);
```

Welke CREATE TABLE moet als
eerst uitgevoerd worden?

```
CREATE TABLE State (  
  state_id INT,  
  state_name TEXT,  
  region TEXT,  
  country TEXT,  
  
  PRIMARY KEY(state_id)  
);
```

“CREATE TABLE State (...)”, want hiernaar wordt verwezen door “Customer”.
Tabellen waarnaar verwezen wordt, moeten al bestaan!

Geldt dit ook voor tabelverwijdering (met DROP TABLE)? Zie Speaker Notes!

Implementatie database-overschrijdende associaties

```
CREATE TABLE SalesOrder_EMPLOYEE (  
  emp_id INT,  
  emp_fname TEXT,  
  emp_lname TEXT,  
  salary FLOAT,  
  dept_id,  
  hrm_employee INT NOT NULL
```

```
  PRIMARY KEY(emp_id),  
  FOREIGN KEY(hrm_employee) REFERENCES  
  HRM_EMPLOYEE(emp_id)  
);
```

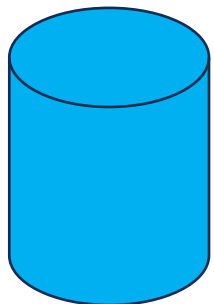
```
CREATE TABLE HRM_EMPLOYEE (  
  emp_id INT,  
  emp_fname TEXT,  
  emp_lname TEXT,  
  salary FLOAT,  
  dept_id INT NOT NULL,  
  salesorder_employee INT NOT NULL
```

```
  PRIMARY KEY(emp_id),  
  FOREIGN KEY(dept_id) REFERENCES  
  Onderhoud Filiaal(filiaalnr),  
  FOREIGN KEY(salesorder_employee)  
  REFERENCES SalesOrder_EMPLOYEE (emp_id)
```

Associaties binnen een database leiden tot een vreemde sleutel.
Database-overschrijdende associaties leiden niet tot vreemde sleutels.

Eerste categorisering inlaadstrategieën

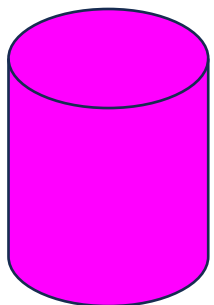
Operationeel



Full Refresh

1	Henk	de Vries
2	Marleen	Langelaan
3	Gerard	van Boven
4	Frida	de Haan

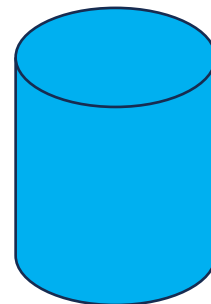
Om wijzigingen bij te houden wordt eerst elke SDM-tabel leeggemaakt, waarna nieuwe data ingeladen wordt.



SDM

1	Henk	de Vries
2	Marleen	Langelaan
3	Gerard	van Boven

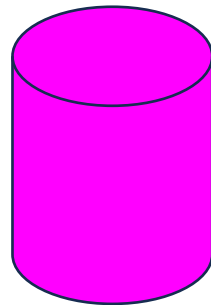
Operationeel



Incremental Data Loading

1	Henk	de Vries
2	Marleen	Langelaan
3	Gerard	van Boven
4	Frida	de Haan

Alléén gewijzigde rijen worden overgezet. Dit geldt niet alleen voor toegevoegde, maar ook voor geüpdate en verwijderde rijen.



SDM

1	Henk	de Vries
2	Marleen	Langelaan
3	Gerard	van Boven

Codevoorbeeld Full Refresh

```
alle_tabellen = [  
    'product',  
    'state',  
    'customer',  
    'salesorder_employee',  
    'sales_order',  
    'sales_order_item',  
    'department',  
    'hrm_employee',  
    'bonus'  
]  
  
alle_tabellen = alle_tabellen[::-1]  
#Draai de lijst om. De laatste toegevoegde tabel moet als eerste worden leeggemaakt.
```

```
for tabel in alle_tabellen:  
    delete_statement = f"DELETE FROM {tabel};"  
  
    try:  
        aenc_sdm_cursor.execute(delete_statement)  
    except pyodbc.Error:  
        print(f"FAILED: {delete_statement}")  
        continue  
  
aenc_sdm_conn.commit()
```

Let op de juiste tabelvolgorde: verwijder eerst de verwijzende tabellen die vreemde sleutels bevatten en daarnaar pas de tabellen waarnaar verwezen wordt.

Probeer de delete-query te executen.
Lukt dat niet? Print de query dan uit.

Maak je wijzigingen (nieuwe inserts) definitief met `.commit()`. Na alle commits moet je de cursor ook weer "closen"

Codevoorbeeld Incremental Data Loading

Producttabel in operationele database:

	id	name	description
8	601	Sweatshirt	Zipped Sweatshirt
9	700	Shorts	Cotton Shorts
10	1000	NEW NAME	NEW DESCRIPTION

In de operationele database is er een nieuw product bijgekomen.

Hoe voeg je alléén dat nieuwe product toe aan het SDM?

Producttabel in SDM:

	id	name	description
8	601	Sweatshirt	Zipped Sweatshirt
9	700	Shorts	Cotton Shorts

Maak een nieuw dataframe met daarin alle verschillen tussen SDM en operationele database

“indicator = True” creëert een kolom “_merge” die per rij 3 mogelijke waarden kan bevatten:

- “both”: de rij komt voor in beide tabellen
- “left_only”: de rij komt alleen voor in de linkertabel
- “right_only”: de rij komt alleen voor in de rechtertabel

```
diff = pd.merge(
    operational_df, #De linkertabel bij deze JOIN
    sdm_df, #De rechtertabel bij deze JOIN
    on='id', #De kolom waarop de JOIN wordt uitgevoerd
    how='outer',
    indicator=True,

    suffixes=('_operational', '_sdm')
    #Bij dubbele kolomnamen worden de suffixes achter de kolomnamen geplakt.
    #Let op: de eerste waarde moet achter de kolommen uit de linkertabel komen,
    #de tweede achter de kolommen uit de rechtertabel.
)
```

```
diff.loc[diff['_merge'] != 'both', :]
```

✓ 0.0s

	id	name_operational	description_operational	name_sdm	description_sdm	_merge
0	1000	NEW NAME	NEW DESCRIPTION	NaN	NaN	left_only

Tweede categorisering inlaadstrategieën

String-built SQL

INSERT	1	Henk	de Vries
INSERT	2	Marleen	Langelaan
INSERT	3	Gerard	van Boven
INSERT	4	Frida	de Haan

Je construeert per rij een SQL-query en stuurt deze vervolgens meteen naar het Datawarehouse.

1	Henk	de Vries
2	Marleen	Langelaan
3	Gerard	van Boven

Schema-Driven Bulk Loading

1	Henk	de Vries
2	Marleen	Langelaan
3	Gerard	van Boven
4	Frida	de Haan

INSERT

Je groepeert rijen eerst in zgn. "batches". Per batch is er vervolgens één query om alle waarden toe te voegen.

1	Henk	de Vries
2	Marleen	Langelaan
3	Gerard	van Boven

Codevoorbeeld String-built SQL

```
for _, row in product_salesorder.iterrows():
    insert_statement = f"INSERT INTO Product VALUES ("

    insert_statement += str(row['id']) + ", "
    insert_statement += str('\'' + row['name'] + '\') + ", "
    insert_statement += str('\'' + row['description'] + '\') + ", "
    insert_statement += str('\'' + row['prod_size'] + '\') + ", "
    insert_statement += str('\'' + row['color'] + '\') + ", "
    insert_statement += str(row['quantity']) + ", "
    insert_statement += str(row['unit_price']) + ", "
    insert_statement += str('\'' + row['picture_name'] + '\') + ", "
    insert_statement += str('\'' + row['Category'] + '\') + ", "
    insert_statement += str('\'' + row['TRIAL814'] + '\') + ", "

    insert_statement = insert_statement[:-2]
    insert_statement += ");"
    # print(insert_statement)

    try:
        aenc_sdm_cursor.execute(insert_statement)
    except:
        print(f"FAILED: {insert_statement}")

aenc_sdm_conn.commit()
```

Let extra op het plaatsen van apostrofjes (') bij deze methode! Deze moeten namelijk *niét* om getallen, maar *wél* om niet-numerieke waarden!

Probeer de insert-query te executen.

Lukt dat niet? Print de query dan uit.

Maak je wijzigingen (nieuwe inserts) definitief met `.commit()`. Na alle commits moet je de cursor ook weer "closen"

Codevoorbeeld Schema-Driven Bulk Loading

```
engine = create_engine("YOUR_CONNECTION_STRING", future=True)
```

```
metadata = MetaData()  
product_table = Table("Product", metadata, autoload_with=engine)
```

```
records = []  
for _, row in product_salesorder.iterrows():  
    records.append({  
        "id": int(row["id"]),  
        "name": row["name"],  
        "description": row["description"],  
        "prod_size": row["prod_size"],  
        "color": row["color"],  
        "quantity": int(row["quantity"]),  
        "unit_price": row["unit_price"],  
        "picture_name": row["picture_name"],  
        "Category": row["Category"],  
        "TRIAL814": row["TRIAL814"],  
    })
```

```
with engine.begin() as conn: # auto-commit/rollback  
    conn.execute(product_table.insert(), records)
```

Met behulp van het MetaData- en het Table-object wordt de tabelstructuur van de te vullen SDM-tabel uitgelezen.

In dit codevoorbeeld is er 1 batch met de naam "records".

In plaats van zelf SQL-code per rij te genereren, laat je sqlalchemy achter de schermen SQL-code per batch generen.

Vergelijking inlaadstrategieën

	String-built SQL	Schema-driven Bulk Loading
Full Refresh	- Eenvoud: makkelijk te debuggen - Snelheid: heel langzaam. Onveranderde data wordt telkens opnieuw ingeladen én dit gebeurt rij voor rij.	- Eenvoud: makkelijk te debuggen - Snelheid: langzaam. Onveranderde data wordt nog steeds (overbodig opnieuw) ingeladen, maar nu met 1 SQL-query per batch, i.p.v. per individuele rij.
Incremental Data Loading	- Eenvoud: moeilijk te debuggen. Je ziet lastiger welke individuele rijen misgaan. - Snelheid: snel, omdat er alléén naar gewijzigde data gekeken wordt.	- Eenvoud: moeilijk te debuggen. Je ziet lastiger welke individuele rijen misgaan. - Snelheid: heel snel, omdat er alléén naar gewijzigde data gekeken wordt én er 1 SQL-query per batch is, i.p.v. per individuele rij.

Ontwikkeling

Productie

Kwadrant
linksbovenKwadrant
rechtsbovenKwadrant
linksonderKwadrant
rechtsonder

print() vs. logging met loguru

```
try:  
    aenc_sdm_cursor.execute(insert_statement)  
except:  
    print(f"FAILED: {insert_statement}")
```

```
aenc_sdm_conn.commit()
```

✓ 0.0s

```
FAILED: INSERT INTO Product VALUES (300, 'Tee Shirt', 'Tank Top', 'Small', 'White', 18, 9, 'tshirtw.bmp', 'Clothes', 'T');  
FAILED: INSERT INTO Product VALUES (301, 'Tee Shirt', 'V-neck', 'Medium', 'Orange', 39, 14, 'tshirto.bmp', 'Clothes', 'T');  
FAILED: INSERT INTO Product VALUES (302, 'Tee Shirt', 'Crew Neck', 'One size fits all', 'Black', 72, 14, 'tshirtb.bmp', 'Clothes', 'T');  
FAILED: INSERT INTO Product VALUES (400, 'Baseball Cap', 'Cotton Cap', 'One size fits all', 'Black', 92, 9, 'capb.bmp', 'Accessories', 'T');  
FAILED: INSERT INTO Product VALUES (401, 'Baseball Cap', 'Wool cap', 'One size fits all', 'White', 12, 10, 'capw.bmp', 'Accessories', 'T');  
FAILED: INSERT INTO Product VALUES (500, 'Visor', 'Cloth Visor', 'One size fits all', 'White', 36, 7, 'visorw.bmp', 'Accessories', 'T');  
FAILED: INSERT INTO Product VALUES (501, 'Visor', 'Plastic Visor', 'One size fits all', 'Black', 28, 7, 'visorb.bmp', 'Accessories', 'T');  
FAILED: INSERT INTO Product VALUES (600, 'Sweatshirt', 'Hooded Sweatshirt', 'Large', 'Green', 39, 24, 'sshirtg.bmp', 'Clothes', 'T');  
FAILED: INSERT INTO Product VALUES (601, 'Sweatshirt', 'Zipped Sweatshirt', 'Large', 'Blue', 32, 24, 'sshirtb.bmp', 'Clothes', 'T');  
FAILED: INSERT INTO Product VALUES (700, 'Shorts', 'Cotton Shorts', 'Medium', 'Black', 80, 15, 'shortsb.bmp', 'Clothes', 'T');
```

print(): genereert slechts tijdelijke output. Deze is alleen zichtbaar totdat de betreffende codecel opnieuw gerund wordt.

Om meerdere runs met elkaar te vergelijken en makkelijker op te sporen welke rijen niet goed inladen, wil je een geschiedenis bijhouden d.m.v. logging.

print() vs. logging met loguru

```
from loguru import logger
logger.add("sdm_log.log")
```

```
try:
    aenc_sdm_cursor.execute(insert_statement)
except:
    # print(f"FAILED: {insert_statement}")
    logger.error(f"FAILED: {insert_statement}")
```

```
aenc_sdm_conn.commit()
```

✓ 0.0s

```
2026-02-01 23:25:09.251 | ERROR | __main__:<module>:26 - FAILED: INSERT INTO Product VALUES (300, 'Tee Shirt', 'Tank Top', 'Small', 'White', 18, 9, 'ts
2026-02-01 23:25:09.255 | ERROR | __main__:<module>:26 - FAILED: INSERT INTO Product VALUES (301, 'Tee Shirt', 'V-neck', 'Medium', 'Orange', 39, 14, 'ts
2026-02-01 23:25:09.256 | ERROR | __main__:<module>:26 - FAILED: INSERT INTO Product VALUES (302, 'Tee Shirt', 'Crew Neck', 'One size fits all', 'Black
2026-02-01 23:25:09.257 | ERROR | __main__:<module>:26 - FAILED: INSERT INTO Product VALUES (400, 'Baseball Cap', 'Cotton Cap', 'One size fits all', 'Bl
2026-02-01 23:25:09.257 | ERROR | __main__:<module>:26 - FAILED: INSERT INTO Product VALUES (401, 'Baseball Cap', 'Wool cap', 'One size fits all', 'Whit
2026-02-01 23:25:09.260 | ERROR | __main__:<module>:26 - FAILED: INSERT INTO Product VALUES (500, 'Visor', 'Cloth Visor', 'One size fits all', 'White',
2026-02-01 23:25:09.261 | ERROR | __main__:<module>:26 - FAILED: INSERT INTO Product VALUES (501, 'Visor', 'Plastic Visor', 'One size fits all', 'Black
2026-02-01 23:25:09.261 | ERROR | __main__:<module>:26 - FAILED: INSERT INTO Product VALUES (600, 'Sweatshirt', 'Hooded Sweatshirt', 'Large', 'Green',
2026-02-01 23:25:09.262 | ERROR | __main__:<module>:26 - FAILED: INSERT INTO Product VALUES (601, 'Sweatshirt', 'Zipped Sweatshirt', 'Large', 'Blue',
2026-02-01 23:25:09.263 | ERROR | __main__:<module>:26 - FAILED: INSERT INTO Product VALUES (700, 'Shorts', 'Cotton Shorts', 'Medium', 'Black', 80, 15,
```

Loguru is één van de meest gangbare “logging”-libraries in Python.

Met deze code wordt alle feedback die je laat genereren opgevangen in een extern .log-bestand.

Om meerdere runs met elkaar te vergelijken en makkelijker op te sporen welke rijen niet goed inladen, wil je een geschiedenis bijhouden d.m.v. logging.

Logging-niveaus

logger.info()

“Waar zijn we in ons proces?”

- Geeft aan wanneer fasen starten en eindigen.
- Niet te gedetailleerd: slechts algemene info, i.p.v. info per coderegel.



logger.warning()

“Wat zou er mis kunnen gaan?”

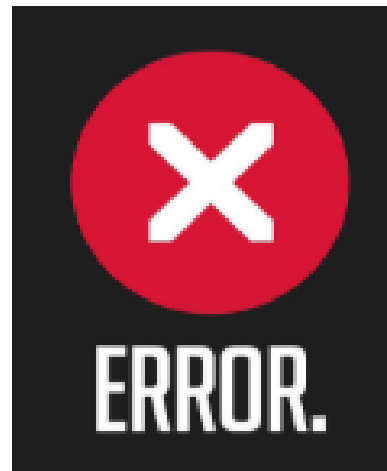
- Geeft weer welke onverwachte resultaten in latere fasen kan optreden en oorzaken daarvan.
- Een soort toekomstvoorspelling.



logger.error()

“Waar faalde ons proces?”

- Signaleert incidenten en alerts.
- De pijn loopt wel door!



logger.debug()

“Wat deed de code precies?”

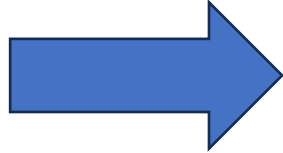
- Geeft toestanden van variabelen en genomen beslissingen op een bepaald moment.
- Vaak slechts tijdelijk, om oorzaken van errors op te sporen!



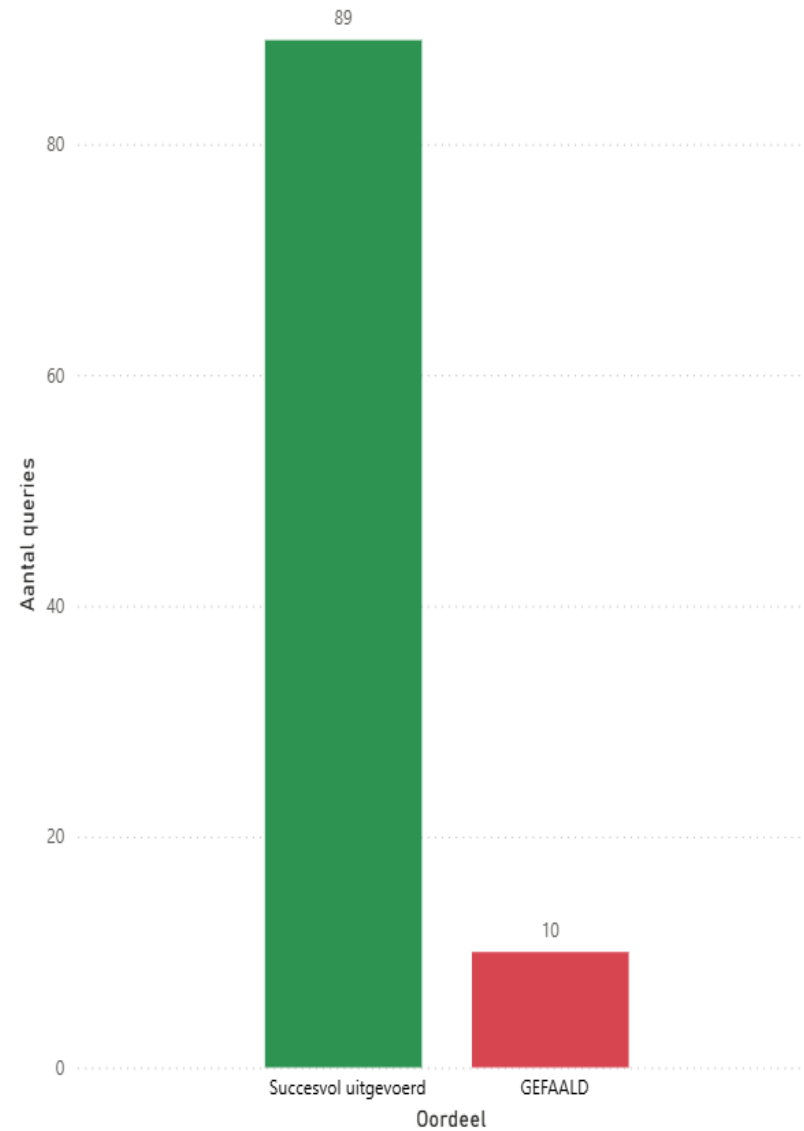
Logdata visualiseren



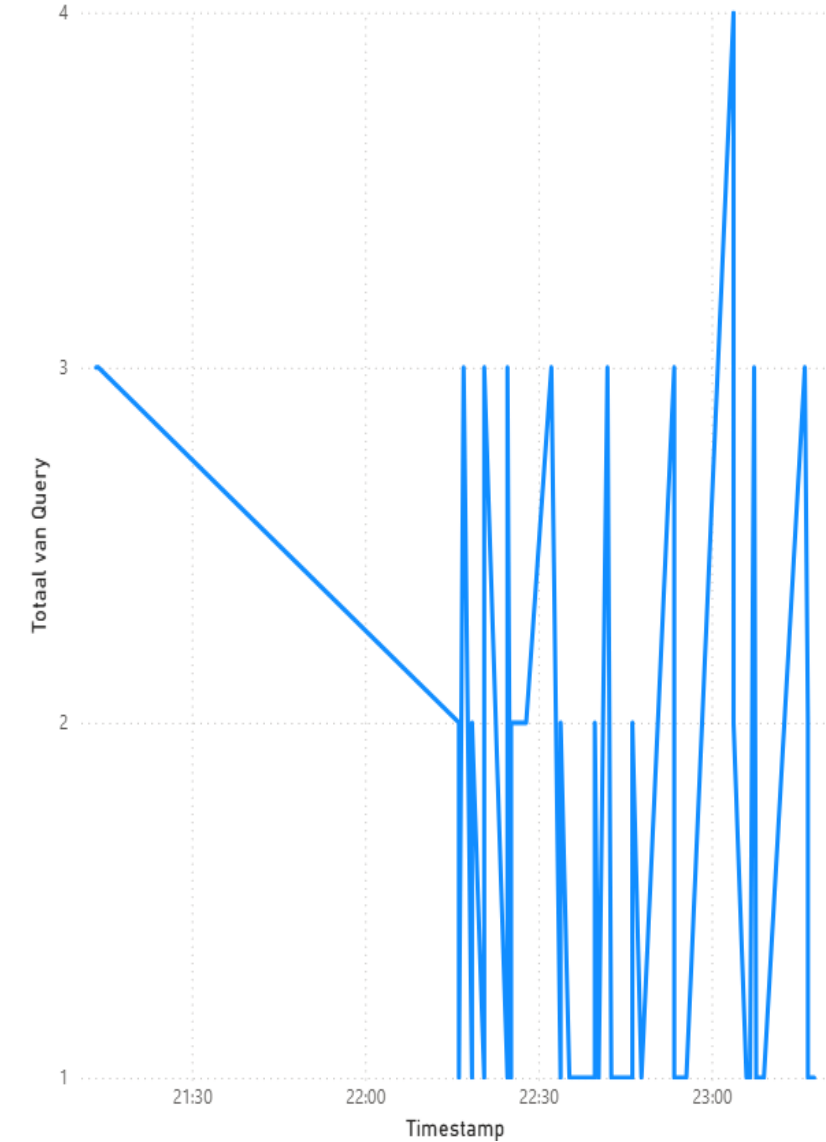
log



Aantal queries per categorie



Verwerkte queries per tijdstip



Meer hierover in de vijfde lesweek van Data Engineering, met behulp van Power BI

Na vandaag...

Weet/ken je...

Kan je...

De juiste CREATE-TABLE-
volgorde bij het aanmaken van
Source Data Modellen

De datastructuur van een
Source Data Model
implementeren in SQLite.

Op welke 4 manieren je
SQLite-databases kan vullen
met behulp van Python

Welke criteria je tegen elkaar
moet afwegen bij het vullen
van Source Data Modellen

Welke logging-niveaus worden
onderscheiden door de
“loguru” library in Python

HC + WC



Source Data Modellen vullen
met data en de gekozen
inlaadstrategie verantwoorden

Logging aanbrengen in
Pythonscripts die Source Data
Modellen vullen vanuit
operationele databases